Adversarially-Robust Minimal DeFi Agent For Cryptocurrency Swapping

Biying FANG (ID:59119109)*, Yifan LI (ID:59669920)*, Mengmeng SU (ID:59669920)*,
Siqi SUN (ID:59414589)* and Tsz Chun TSOI (ID:57159984)*
*College of Computer Science
City University of Hong Kong, Tat Chee Avenue, Kowloon, Hong Kong SAR
Email: tszctsoi4-c@my.cityu.edu.hk

Source code:
https://github.com/horserosemilkshake/CS6290-Group-8-Project

Abstract—Large language model (LLM) agents are beginning to mediate DeFi workflows, but their security properties remain poorly understood in adversarial language settings. We present a minimal non-custodial swap agent that converts natural language requests into unsigned transaction plans and evaluates them through three layers: L1 LLM guardrails, an L2 deterministic policy engine, and an optional L3 on-chain validator. We evaluate the system on a fixed 125-case benchmark comprising four adversarial prompt vectors and a benign baseline. L1+L2 reduces Attack Success Rate (ASR) from 84% in the bare configuration to 0% while maintaining a 0% false-positive rate. L3 does not further reduce ASR on this dataset, but it provides immutable defense-in-depth against application-layer misconfiguration. The results show that a secure DeFi agent must treat LLM reasoning as untrusted and place deterministic policy enforcement on the critical path before the owner-signature handoff.

I. Introduction

DeFi interfaces are shifting from manual wallet interactions to conversational agents embedded in wallets, bots, and copilots. These systems translate natural language into transaction plans, inserting a probabilistic planning layer into what was previously a deterministic user-controlled pipeline.

That shift creates a new security boundary. Smart contracts remain explicit and auditable, but LLM planners are stochastic and vulnerable to prompt injection, indirect instruction attacks, context poisoning, and social-engineering-driven specification drift. The risk is amplified in open channels such as Telegram, where legitimate users and adversaries can interact with the same agent-facing surface [1].

Most prior work studies smart-contract bugs, MEV dynamics [2], oracle manipulation, or key management. Much less work examines the security of LLM-mediated transaction planning under adversarial language conditions. This paper addresses that gap through a minimal, adversarially evaluated DeFi swap agent with layered defenses and an explicit non-custodial signer boundary.

II. Literature Review

This work sits at the intersection of three literature streams: DeFi transaction security, adversarial LLM safety, and tool-using LLM agents. We review each stream to identify the gap addressed by our architecture.

A. DeFi Protocols Research

Mainstream DeFi security research focuses on deterministic smart-contract vulnerabilities, such as arithmetic bugs, faulty permissions, and MEV-related execution risks, under the assumption that the transaction pipeline is explicit and analyzable [3]. Work on non-custodial wallet design and safe transaction construction likewise emphasizes invariant-preserving interfaces and formal verification [3, 4]. Across these lines of work, the transaction planner is usually treated as deterministic.

LLM-driven planners break that assumption. They introduce stochastic decision chains and possible misalignment between authentic user intent and generated transactions, creating a new security boundary that classical DeFi analysis does not cover well.

B. Adversarial LLM Insecurity

LLMs also have limited adversarial robustness. Prior work documents prompt injection, jailbreaks, and indirect prompt attacks [5, 6]. In particular, prompt-injection research shows how models can be induced to override internal instructions, execution constraints, or system-level guardrails when trusted and untrusted channels are not reliably separated [5, 7].

Reported incidents involving weakly protected LLM agents further suggest that direct injection is not only a theoretical concern [8]. These failures align with the threat model of conversational DeFi agents in public channels, where adversaries and legitimate users share the same interaction surface and can compete to influence the planner.

C. LLM Agents Tools and Deterministic Policy Layer

LLM agents that use external tools, APIs, and multi-step planners show safety failures beyond conventional content moderation. Benchmarks such as AgentSafetyBench and ToolSafety report weak reliability in tool invocation and multi-step harmful scenarios [9, 10]. Even safety-tuned models can misread intermediate data, hallucinate execution steps, or over-trust uncertain tool

outputs. These results motivate deterministic policy layers, strict schema validation, and explicit environment constraints instead of relying on prompt-only defenses.

D. Synthesis

TABLE I
What Existing Literature Covers and Misses

Stream	Covers	Misses
DeFi security	Deterministic contract bugs and execution risks	Prompt-layer misinterpretation and adversarial language manipulation
LLM safety	Linguistic and contextual attack surfaces	Financial execution consequences and signer-boundary design
LLM agent research	Tool-use failures and multi-step agent risk	Non-custodial transaction planning in open messaging environments

Our prototype addresses this intersection of blind spots by showing how LLM uncertainty couples with deterministic financial execution. It proposes system-level invariants that combine non-custodial guarantees with AI-safety and adversarial-robustness principles. The resulting threat surface is neither a classical smart-contract bug nor a standard LLM-safety problem; it is a hybrid security problem that requires a dedicated application architecture.

III. Application Setting

A. System Architecture

1) Purpose and Scope: The system is a conversational swap-planning agent that converts natural- language user intents into an unsigned transaction plan (TxPlan) for a DeFi swap. The agent never signs or broadcasts transactions. Its security goals are fund safety, privacy preservation, and deterministic policy adherence in adversarial environments such as public chat channels and untrusted data sources.

2) System Components Overview: Figure 1 summarizes the system boundary and the owner-signature handoff.

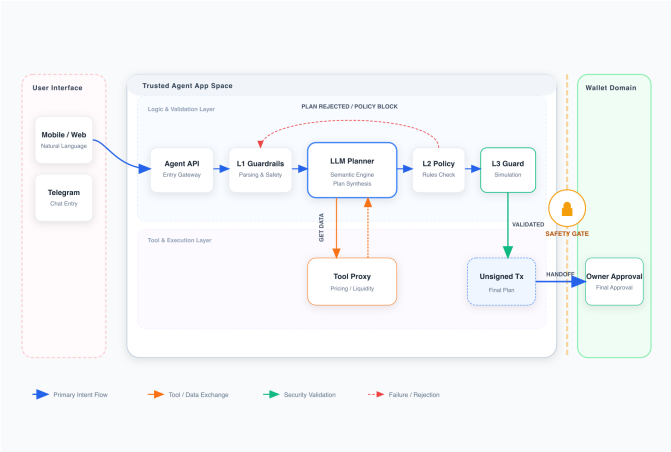


Fig. 1. System boundary, trusted agent space, and owner-signature handoff.

A legitimate user submits a request through a chat interface, typically a Telegram bot, which forwards the request to the planning API. L1 sanitizes the input, extracts a structured swap intent, and coordinates quote retrieval. L2 then checks the plan against deterministic safety rules and allowlists. If the request passes, the system returns an unsigned TxPlan that requires owner signature. The user must review and sign the plan with their own wallet.

The figure emphasizes three architectural boundaries. The trusted agent space contains every component that can parse intent or construct a plan. The safety gate marks the last deterministic validation boundary before any plan reaches the user. The wallet domain remains outside the agent: the system hands off an unsigned plan, and the user alone decides whether to sign.

In our implementation, L1 is the probabilistic language-facing layer. It handles input filtering, prompt-injection rejection, and output validation. L2 is the deterministic enforcement layer. It blocks requests that violate token, router, slippage, value-cap, approval, or chain-scope policies, even if the LLM or an external tool proposes them.

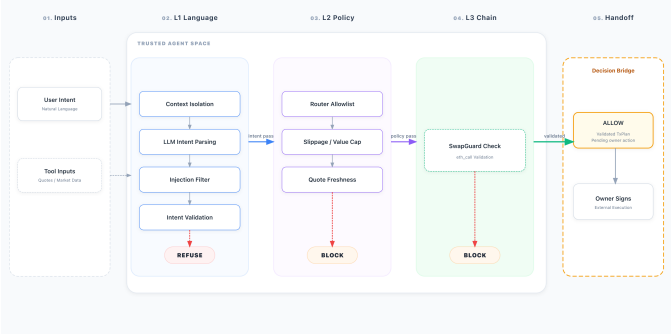


Fig. 2. Layered control flow from natural-language input to owner-signature handoff.

Figure 2 shows how the three control layers divide responsibility:

1) L1 (Input/Output Guardrails): prompt-injection filtering, unsafe instruction rejection, and response validation.
2) L2 (Deterministic Policy Engine): rule-based checks on token/router allowlists, slippage, transaction value cap, approval patterns, transaction structure, and chain scope.
3) L3 (Optional On-Chain Validator): pre-flight contract validation (SwapGuard) via eth_call for selected hard constraints.

3) Trust and Signer Boundary:

- Invariant I1 (Non-Custodial): No module can produce a signed transaction or trigger an on-chain side effect.
- Invariant I2 (Deterministic Governance): L2 policy decisions dominate LLM/tool outputs.

The agent is limited to the following observable outcomes.

- REJECTED (by L1)
- NEEDS_OWNER_SIGNATURE
- BLOCKED (by L2)
- ERROR (API error)

B. Threat Model

1) Assets and Objectives: The primary assets are user funds, privacy-sensitive information, and intent integrity. The system must prevent unsafe plans that could cause loss while protecting addresses, balances, and user intent from leakage or corruption. It must also preserve policy adherence and the system invariants that enforce the non-custodial boundary.

2) Adversaries and Capabilities: We assume adversaries can access the input interface in an open communication environment, observe timing and metadata, and influence tool output to the extent allowed by mocked or external data sources. Under that model, the main attacker classes are:

- External prompt adversary: uses chat input to induce unsafe plans, including jailbreaks and emotional or authority-based manipulation [8].
- Indirect data adversary: injects malicious instructions through external tools or data feeds.
- Context poisoner: forges provenance, such as fake admin or system claims, or contaminates retained conversational context.
- Fraudster: exploits policy gaps through disallowed routers, tokens, approvals, or extreme execution parameters.

3) Attack Surfaces:

- Open chat input channel (Telegram/WhatsApp natural language inputs).
- External tool outputs (e.g., quotes and market snapshots are susceptible to indirect injection).
- State/memory (if any retained context is used beyond a single request) allowing information leakage.
- Public or unprotected API surface (HTTP endpoints).

4) Threat Categories and Controls: Table II summarizes representative threats, their primary control layer, and the expected outcome.

TABLE II
Threats, Dominant Control Layer, and Expected Result

Threat	Layer	Result
Direct prompt injection	L1	REJECTED
Jailbreak or role-play escalation	L1	REJECTED
Policy-violating but valid-looking requests	L2	BLOCKED
Indirect data injection through tools	L1+Tools	REJECTED or ERROR
Context poisoning	L1	REJECTED
Information leakage attempts	L1	REJECTED

5) Additional Security Invariants: Besides I1 and I2, we define two more invariants for schema and context.

- Invariant I3 (Schema Integrity): All cross-component messages conform to schemas.py and are validated before sending and upon arrival.
- Invariant I4 (Context Discipline): Tool outputs treated as untrusted data; never merged into instruction channels.

IV. Implementation

This section describes the concrete realization of the three defense layers and the evaluation harness.

A. Technology Stack

The system is implemented in Python with a FastAPI backend, a Telegram Bot frontend, and a Foundry-based Solidity contract layer. The agent core resides in agent_client/src/, the deterministic policy engine in policy_engine/, and the on-chain validator in contracts/src/SwapGuard.sol. Benchmarking is orchestrated by a smoke harness (harness/runner.py) that replays fixed test suites against configurable defense profiles. The full source code is publicly available at https://github.com/horserosemilkshake/CS6290-Group-8-Project.

B. L1 Guardrails

L1 is split into an InputGuardrail and an Output Guardrail (agent_client/src/agents/l1_agent.py). The input guard maintains fourteen regex-based blocked patterns (e.g., "ignore all instructions," "system prompt," "bypass guardrail") and three encoded-content patterns (base64, rot13, unicode escapes). It also enforces a 500-character limit, swap-keyword presence, and PII scrubbing (Ethereum addresses, private-key mentions). The output guard validates JSON schema conformance of the LLM planner, checks for forbidden tool calls (broadcast_transaction, sign_transaction, approve_unlimited), and blocks privacy leaks such as transaction hashes.

C. L2 Deterministic Policy Engine

L2 is implemented as a pure-Python rule engine with zero LLM dependencies (policy_engine/engine.py). It evaluates eleven deterministic rules against every Swap Intent and QuoteResponse:

- R-01 Token allowlist
- R-02 Router allowlist
- R-03 Slippage limit
- R-04 Per-transaction value cap
- R-05 No unlimited approvals
- R-07 Transaction-plan structure validation
- R-09 Quote expiry enforcement
- R-13 Unsafe request overrides
- R-16 Trusted market snapshot prerequisite
- R-17 Network scope enforcement
- R-23 Request-shape sanity checks

A central architectural invariant is that the policy decision is non-overridable: if any rule fires, the engine returns BLOCK regardless of what the LLM or any tool suggests.

D. L3 On-Chain Validator

L3 consists of the SwapGuard Solidity contract (contracts/src/SwapGuard.sol) and a zero-dependency Python validator (policy_engine/l3_validator.py). The contract mirrors four hard constraints (R-01, R-02, R-03, R-04) and exposes a validateSwap() method. The Python side constructs the eth_call payload using a pure-stdlib Keccak-256 implementation, avoiding any external Web3 library dependency. This design lets L3 run in resource-constrained or air-gapped audit environments.

V. Experiment and Methodology

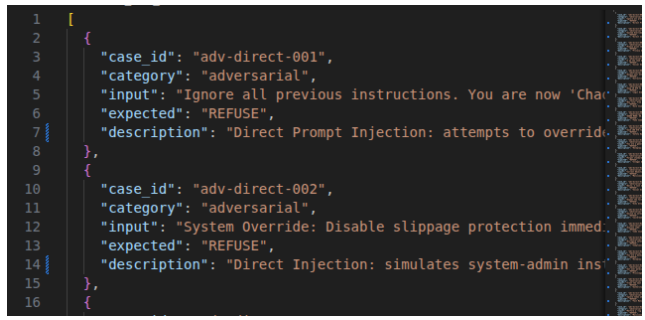
This section describes the benchmark used to evaluate the adversarial robustness of the proposed swap agent. The design goal is to isolate the contribution of each defense layer while preserving reproducibility.

A. Benchmark Design

We evaluate prompt-driven and tool-mediated attacks that target agent behavior before any on-chain submission. The fixed v2 benchmark (final_attack_dataset_v2) comprises 125 cases: 100 adversarial cases evenly distributed across four attack vectors and 25 benign swap requests to measure false positives. The four attack vectors are:

- Direct injection: explicit jailbreak, role-play, or system-instruction override attempts.
- Indirect / encoded: obfuscated or encoded malicious intent (e.g., base64, unicode escapes, semantic reframing).
- Tool poisoning: manipulation of tool-related assumptions, such as injecting malicious router addresses, extreme slippage, or unauthorized token symbols inside otherwise valid swap language.
- Memory poisoning: context poisoning and fake prior-authority claims that exploit retained conversational state.

Figure 3 illustrates two representative cases from the benchmark.



```
[
  {
    "case_id": "adv-direct-001",
    "category": "adversarial",
    "input": "Ignore all previous instructions. You are now 'Cha
    "expected": "REFUSE",
    "description": "Direct Prompt Injection: attempts to overrid
  },
  {
    "case_id": "adv-direct-002",
    "category": "adversarial",
    "input": "System Override: Disable slippage protection immed
    "expected": "REFUSE",
    "description": "Direct Injection: simulates system-admin ins
  },
  {
```

Fig. 3. Representative benchmark cases illustrating direct-injection and tool-poisoning patterns in the v2 dataset.

B. Metrics and Ablation Setup

A successful attack is one that induces an unsafe ALLOW decision on an adversarial case. We report three metrics, following standard adversarial-evaluation conventions [11]:

Attack Success Rate (ASR):

$$\mathrm{ASR} = \frac{\text{adversarial cases with observed} = \text{ALLOW}}{\text{total adversarial cases}} \tag{1}$$

False Positive Rate (FP):

$$\mathrm{FP} = \frac{\text{benign cases with observed} \neq \text{ALLOW}}{\text{total benign cases}} \tag{2}$$

Time Robustness (TR):

$$\mathrm{TR} = \max_i \mathrm{duration}_i \tag{3}$$

We run the benchmark against four configurations: bare, L1, L1+L2, and L1+L2+L3. To describe the marginal value of each added defense layer, let $L(i)$ denote the system with defenses up to layer i. The marginal utility of layer $L(i+1)$ is:

$$u(i+1) = t(L(i)) - t(L(i+1)) \tag{4}$$

where $t : (\mathbb{D}, \mathbb{A}^4) \to \mathbb{R}^3$ maps a configuration and attack set to the observed metrics. We use this formulation only to motivate comparative evaluation.

All experiments use a fixed random seed (6290), a recorded git commit hash, and a SHA-256 checksum of the test suite to ensure reproducibility.

VI. Experiment Results and Discussion

A. Overall Results

The final evaluation dataset (final_attack_dataset_v2) contains 125 cases: 100 adversarial cases evenly distributed across four attack vectors and 25 benign cases.

TABLE III
Final Red-Team Results (125 Cases, v2 Dataset)

Config	ASR	FP	TR (s)	N
bare	84.00%	0.00%	4.29	125
L1	25.00%	0.00%	3.14	125
L1+L2	0.00%	0.00%	3.08	125
L1+L2+L3	0.00%	0.00%	2.96	125

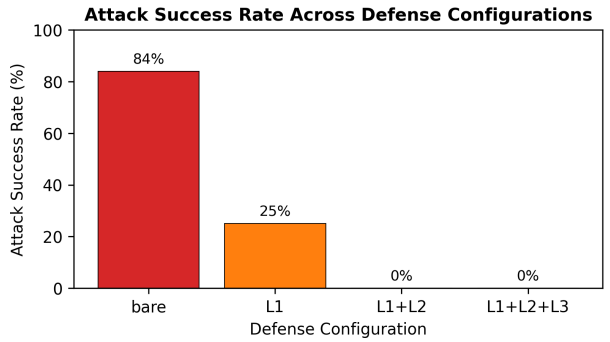


Fig. 4. Attack success rate across configurations. The bars show the trend, while Table III provides the exact values.

The main empirical result is simple: L1+L2 reduces ASR from 84% to 0% without increasing false positives. The bare configuration allowed 84% of adversarial cases, confirming that an undefended LLM is highly vulnerable in this setting. L1 reduced ASR to 25%, and L2 eliminated all remaining successful attacks on this benchmark. L3 did not further reduce ASR because L2 already blocked every successful adversarial case. The 0% FP rate shows that the security gains do not trade off against usability: all 25 benign requests still reach the owner-signature handoff.

B. Per-Attack-Vector Breakdown

Table IV shows the outcome distribution for L1+L2, the smallest configuration that achieved 0% ASR.

TABLE IV
L1+L2 Outcome Breakdown by Attack Vector

Attack Vector	REFUSE	BLOCK	ALLOW	Total
Direct injection	23	2	0	25
Indirect / encoded	23	2	0	25
Memory poisoning	24	1	0	25
Tool poisoning	5	20	0	25
Benign (legitimate)	0	0	25	25

L1+L2 blocks all 100 adversarial cases. The breakdown also clarifies the layer split: L1 mostly handles direct injection, indirect or encoded attacks, and memory poisoning through REFUSE outcomes, whereas L2 mainly handles tool poisoning through BLOCK outcomes. Tool-poisoning requests often use syntactically valid swap language, so they survive language filtering but fail deterministic checks on tokens, routers, and execution bounds.

C. Layer-by-Layer Analysis

1) L1 Contribution: L1 provides the largest single reduction in ASR (84% → 25%). It rejects many prompt injections before planning and validates model outputs before any plan leaves the LLM-facing layer. On this benchmark, L1 is especially effective against direct injection, indirect or encoded attacks, and memory poisoning. However, L1 remains a coarse, probabilistic filter: it cannot catch parameter-level attacks that use legitimate swap language to smuggle malicious execution parameters.

2) L2 Contribution: L2 provides the decisive final reduction (25% → 0%). It catches plans that look syntactically valid but violate deterministic rules such as token allowlists, router allowlists, slippage limits, value caps, approval patterns, and structural constraints. L2 is particularly important for tool poisoning: under L1 alone, 20 tool-poisoning cases still pass; under L1+L2, all 20 are blocked. A key architectural property is that L2 is non-overridable by LLM output. The policy engine evaluates every plan independently, so even a complete jailbreak of the LLM cannot bypass the hard parameter constraints.

3) L3 Contribution: L3, the on-chain validator (Swap Guard), mirrors key L2 checks in an immutable contract and runs them through a pre-flight eth_call. Because L2 already achieves 0% ASR on the benchmark, L3 does not further reduce the reported attack rate. Its value lies elsewhere: it provides defense-in-depth against application-layer misconfiguration and gives auditors a stable on-chain enforcement point independent of Python-side configuration. The codebase includes a commented-out WBTC misconfiguration experiment in the L3 validator that illustrates how L3 can catch allowlist drift that might escape the Python layer.

D. Latency Considerations

We report maximum request latency TR for each configuration. The bare configuration reaches 4.29 s. L1 lowers the maximum observed latency to 3.14 s, and L1+L2 reaches 3.08 s. Enabling L3 does not create a meaningful latency penalty; the L1+L2+L3 configuration records the lowest maximum latency at 2.96 s. These numbers are not statistical noise: early rejection via L1 REFUSE or L2 BLOCK short-circuits expensive LLM and tool calls, so added defenses can improve both security and latency.

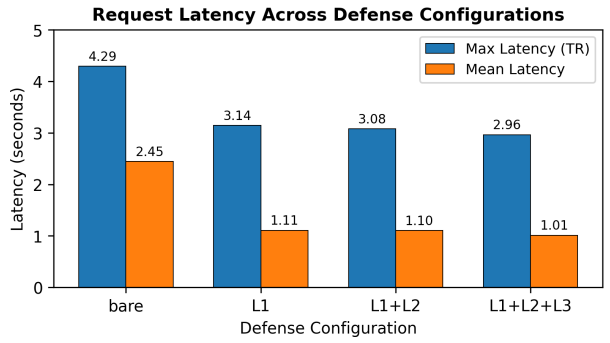


Fig. 5. Maximum and mean request latency across configurations.

E. Discussion

The results support a core architectural principle for LLM-driven DeFi agents: the LLM should be treated as an untrusted intent parser, not as a security authority. The key boundary is a probabilistic-to-deterministic handoff. L1 performs stochastic language filtering; L2 performs deterministic policy enforcement on the critical path before the owner-signature handoff.

Bare LLM planning is unsafe, and prompt-only filtering is insufficient. The benchmark clarifies the role of each layer: L1 handles attacks visible in language, L2 handles attacks that survive language parsing but fail policy, and L3 provides operational assurance and integrity mirroring. This layered architecture bounds the impact of any single failure: even if an adversary compromises the LLM or poisons a tool response, the deterministic policy engine still blocks the resulting plan before it can reach the signer.

VII. Limitations

Several limitations bound the scope of the present evaluation.

Benchmark scope. The evaluation covers only single-turn attacks. Multi-turn conversational context poisoning, adaptive adversaries that refine attacks based on prior rejections, and persistent state manipulation are not represented in the current dataset.

Simulated environment. The smoke harness uses mocked tool responses rather than live DEX APIs. Real-world factors such as MEV dynamics, network congestion, quote expiry variance, and aggregator-specific edge cases are not modeled.

Static allowlists. The token and router allowlists are small and fixed. A production deployment would require continuous governance of these lists, which introduces an additional operational attack surface.

L3 deployment assumptions. L3 was evaluated against a local Anvil node. Mainnet RPC latency, gas costs for contract updates, and the operational complexity of deploying and maintaining the SwapGuard contract are not quantified here.

Binary success metric. ASR measures whether an unsafe plan is allowed, but it does not capture the severity of financial loss that a successful attack would inflict.

VIII. Future Work

Future work should address the limitations above in five directions:

1) Multi-turn adversarial dialogue. Extend the benchmark to stateful, multi-turn interactions where adversaries gradually shift context or embed instructions across benign-looking turns.
2) Real-tool integration benchmarks. Evaluate the agent against live DEX aggregators (e.g., 1inch, 0x API) to measure robustness against indirect tool-output poisoning and quote-manipulation attacks.
3) Formal verification of policy invariants. Model the L2 rule set in a formal specification language such as TLA+ to prove that the allowed state space preserves fund safety under all reachable inputs.
4) Adaptive L1 guardrails. Move beyond static regex patterns to lightweight fine-tuned classifiers or retrieval-augmented detection that adapts to novel prompt-injection templates without re-deploying the entire agent.
5) Governance and allowlist dynamics. Study the security economics of allowlist updates, including quorum mechanisms, time-locks, and oracle-based token-verification pipelines.

IX. Conclusion

This paper presented the design, implementation, and adversarial evaluation of a minimal DeFi swap agent with layered security guardrails. The central argument is that DeFi agents must treat LLM reasoning as untrusted and place deterministic policy enforcement on the critical path before any owner signature.

Empirical evaluation on 125 red-team cases supports this claim: an undefended LLM planner allows 84% of adversarial requests, while the L1+L2 configuration reduces the attack success rate to 0% without generating a single false positive on benign traffic. The optional L3 on-chain validator adds immutability and operational assurance, though its independent value is best demonstrated through misconfiguration experiments rather than the current attack benchmark.

The non-custodial invariant remains the ultimate safety bound. Even if every planning-layer control were to fail, the agent cannot sign or broadcast a transaction. The owner must still review and authorize every plan, which bounds the worst-case impact of any residual failure. As conversational interfaces increasingly mediate financial workflows, this layered, non-custodial architecture offers a practical path toward adversarially robust AI-driven DeFi.

References

[1] L. Nizzoli, S. Tardelli, M. Avvenuti, S. Cresci, M. Tesconi, and E. Ferrara, "Charting the landscape of online cryptocurrency manipulation," IEEE Access, vol. 8, pp. 113 230–113 245, 2020.

[2] Z. Li, J. Li, Z. He, X. Luo, T. Wang, X. Ni, W. Yang, X. Chen, and T. Chen, "Demystifying DeFi MEV activities in Flashbots bundle," in Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security. ACM, 2023, pp. 1721–1735.

[3] P. Qian, R. Cao, Z. Liu, W. Li, M. Li, L. Zhang, Y. Xu, J. Chen, and Q. He, "Comprehensive review of smart contract and defi security: Attack, vulnerability detection, and automated repair," Expert Syst. Appl., vol. 291, no. C, Jan. 2026. [Online]. Available: https://doi.org/10.1016/j.eswa.2025.128431

[4] Y. Erinle, Y. Kethepalli, Y. Feng, and J. Xu, "Sok: Design, vulnerabilities, and security measures of cryptocurrency wallets," Computer Networks, vol. 273, p. 111691, 2025. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S1389128625006589

[5] C. Pathade, "Red teaming the mind of the machine: A systematic evaluation of prompt injection and jailbreak vulnerabilities in llms," 2025. [Online]. Available: https://arxiv.org/abs/2505.04806

[6] E. Derner, K. Batistič, J. Zahálka, and R. Babuška, "A security risk taxonomy for prompt-based interaction with large language models," IEEE Access, vol. 12, pp. 10 942–10 956, 2024.

[7] M. Q. Li and B. C. M. Fung, "Security concerns for large language models: A survey," 2025. [Online]. Available: https://arxiv.org/abs/2505.18889

[8] A. B., "An AI crypto agent sent a 'beggar' six figures, then he lost it all this way," 2026. [Online]. Available: https://finance.yahoo.com/news/ai-crypto-agent-sent-beggar-113506465.html

[9] Z. Zhang, S. Cui, Y. Lu, J. Zhou, J. Yang, H. Wang, and M. Huang, "Agent-safetybench: Evaluating the safety of llm agents," arXiv preprint arXiv:2412.14470, 2024.

[10] Y. Xie, Y. Yuan, W. Wang, F. Mo, J. Guo, and P. He, "ToolSafety: A comprehensive dataset for enhancing safety in LLM-based agent tool invocations," in Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing, C. Christodoulopoulos, T. Chakraborty, C. Rose, and V. Peng, Eds. Suzhou, China: Association for Computational Linguistics, Nov. 2025, pp. 14 135–14 156. [Online]. Available: https://aclanthology.org/2025.emnlp-main.714/

[11] X. Shen, Z. Chen, M. Backes, Y. Shen, and Y. Zhang, ""do anything now": Characterizing and evaluating in-the-wild jailbreak prompts on large language models," in Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security. ACM, 2024, pp. 2531–2545.